Hardware

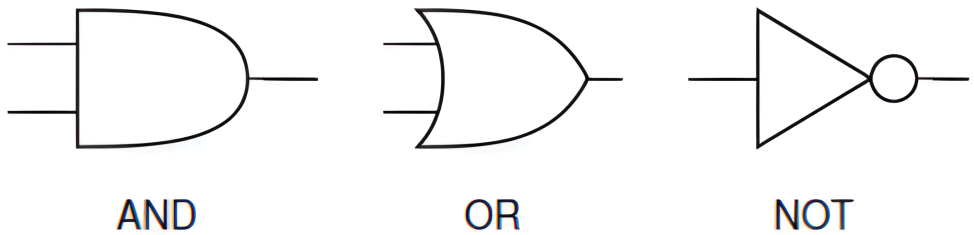
Introduction to computer programming

Management and Artificial Intelligence

Boolean operators as logic gates

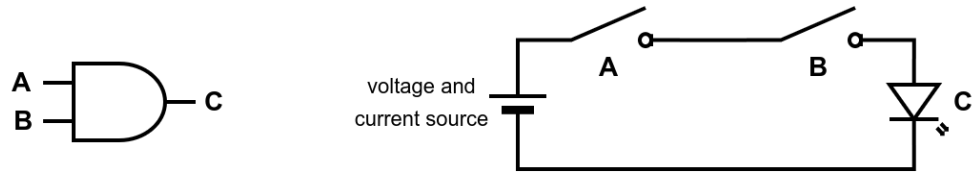
The boolean operators AND, OR, and NOT, introduced in the previous lecture, can be implemented as logic gates (circuits).

Within a circuit, they are represented by these symbols:



AND gate

The AND logic gate can be implemented with two switches in series:



The two inputs, A and B, control the two switches:

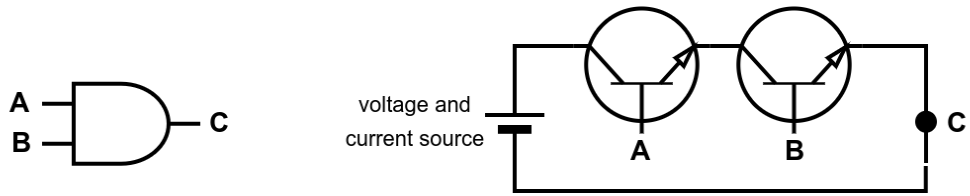
- if their boolean value is True, the switch is closed and current flows
- if their boolean value is False, the switch is open and no current flows

Only when both switches are closed (A=True and B=True), the current flows to C (represented with a LED, which would emit light to represent a True value).

NOTE: A resistor should be added in series with the LED to limit the current and prevent damage.

AND gate: from switches to transistors

In electronics, when realizing an AND gate, switches are replaced with transistors:



We have replaced the two switches with two BJT NPN transistors:

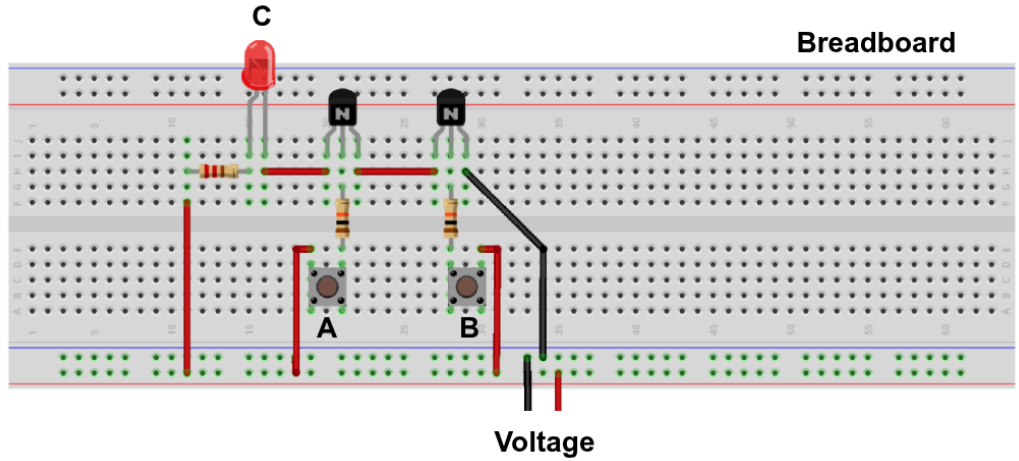
- if their base signal (A or B) is high, the transistor is on and current flows
- if their base signal (A or B) is low, the transistor is off and no current flows

Only when both transistors are on, the output signal C is high.

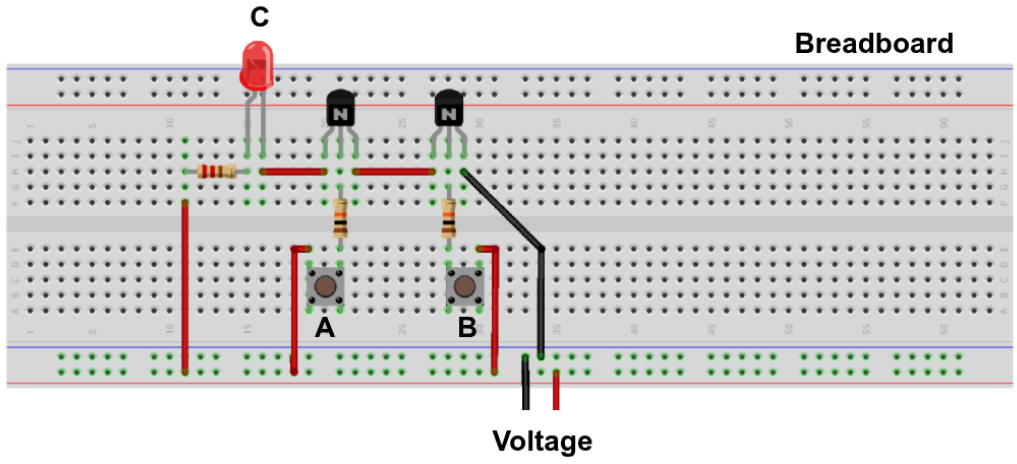
NOTE: There exist other types of transistors, which can be used to implement an AND gate.

AND gate: real-world (inefficient) implementation

We can buy transistors, resistors, and LED to make our own AND gate with minimal cost: "electronics starter kit" (often sold with Arduino boards) cost less than 15-20€ and allows to build simple circuits.



- the breadboard is a device that allows you to connect the components without soldering them
- we added some resistors to limit the current flowing through the LED
- we can manually control A and B by pushing the buttons
- the LED will light up if and only if both A and B are pressed

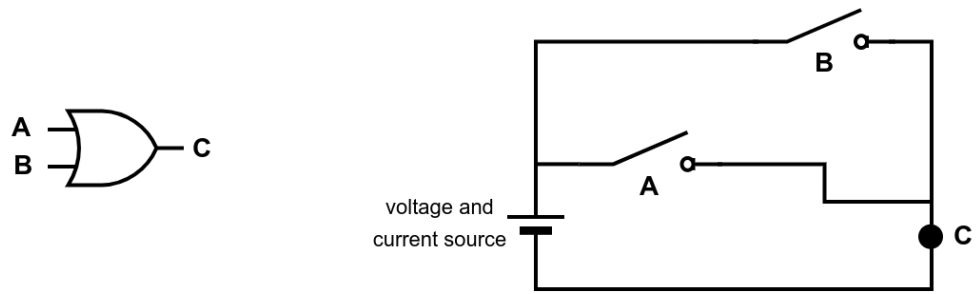


In the real world:

- A and B would be generated by other circuits, without the need of manually controlling them!
- C would be used to likely control other circuit or generate a bit value

OR gate

The OR logic gate can be implemented with two switches in parallel:



The two inputs, A and B, control the two switches:

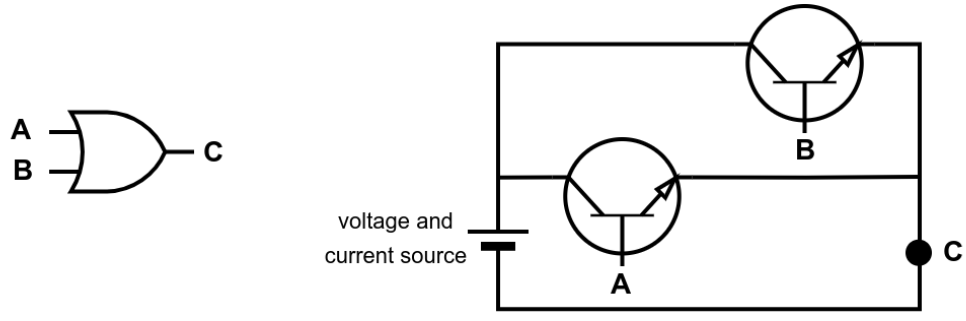
- if their boolean value is True, the switch is closed and current flows
- if their boolean value is False, the switch is open and no current flows

When at least one switch is closed (A=True or B=True), the current flows to C (represented with a LED, which would emit light to represent a True value).

NOTE: A resistor should be added in series with the LED to limit the current and prevent damage.

OR gate: from switches to transistors

In electronics, when realizing an OR gate, switches are replaced with transistors:



We have replaced the two switches with two BJT NPN transistors:

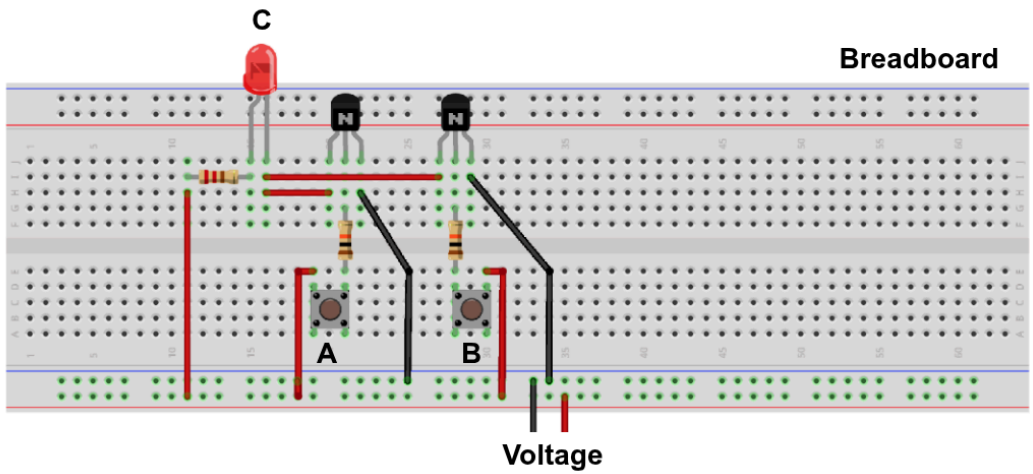
- if their base signal (A or B) is high, the transistor is on and current flows
- if their base signal (A or B) is low, the transistor is off and no current flows

When at least one transistors is on, the output signal C is high.

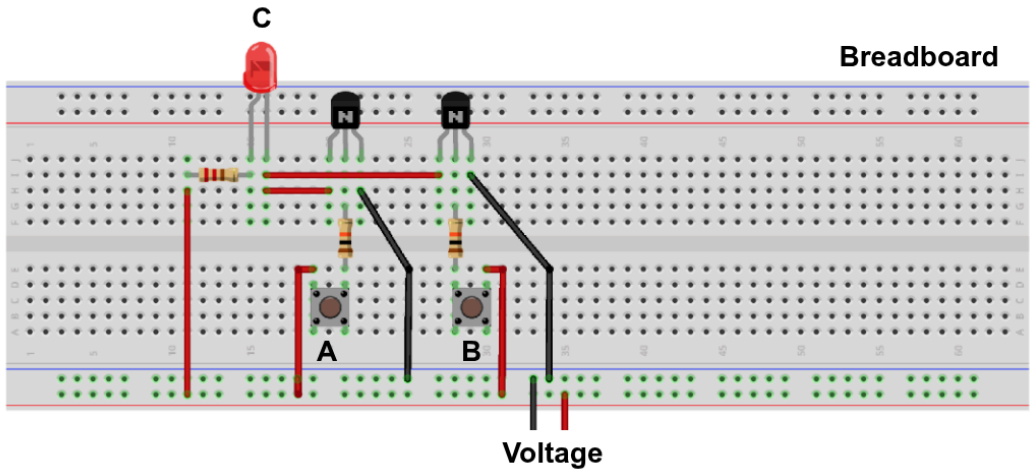
NOTE: There exist other types of transistors, which can be used to implement an AND gate.

OR gate: real-world (inefficient) implementation

As for the AND gate, we can implement the OR gate with a few cheap transistors, resistors, and LED.



- the breadboard is a device that allows you to connect the components without soldering them
- we added some resistors to limit the current flowing through the LED
- we can manually control A and B by pushing the buttons
- the LED will light up if at least one of A or B is pressed



In the real world:

- A and B would be generated by other circuits, without the need of manually controlling them!
- C would be used to likely control other circuit or generate a bit value

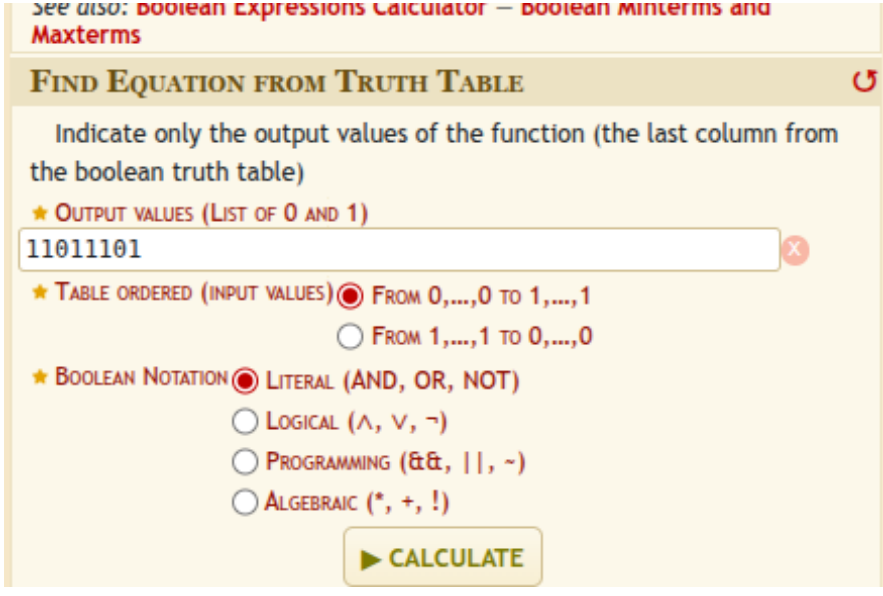
What can we do with AND, OR, NOT gates?

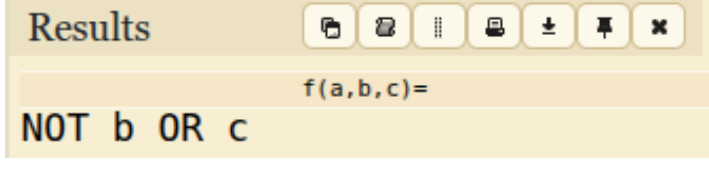
We can combine AND, OR, and NOT gates to implement a logic circuit that implements arbitrary boolean functions:

1. Build the truth table for your custom computation. For instance:

A	B	C	OUTPUT
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

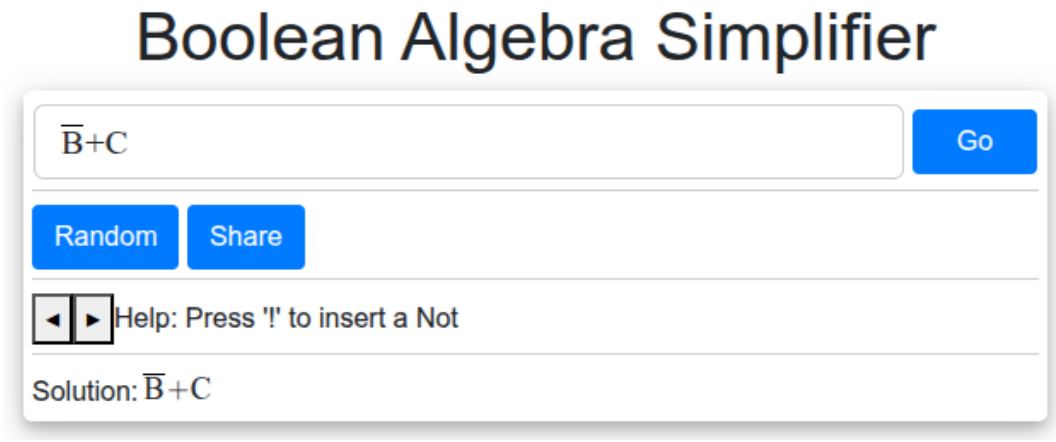
2. Use a tool to derive the boolean formula: e.g., provide the OUTPUT column from your truth table to dcode.fr

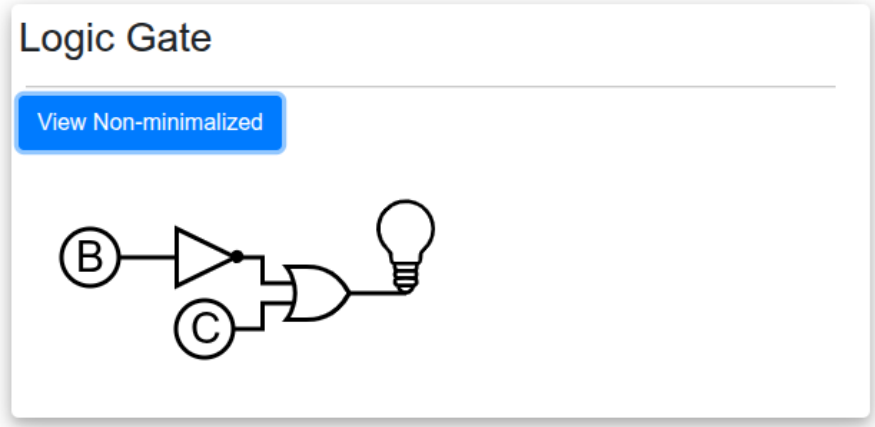




NOTE: We discovered that our output does not depend on the value of A!

3. Use a tool to get a minimal circuit: e.g., provide the boolean formula to boolean-algebra.com





NOTE: In this case, the formula was already minimal. In general, the tool will first perform a minimization of the formula, and then generate the circuit. Smaller is formula, smaller is circuit, less gates, less area, less power, less cost.

Implementing real-world components

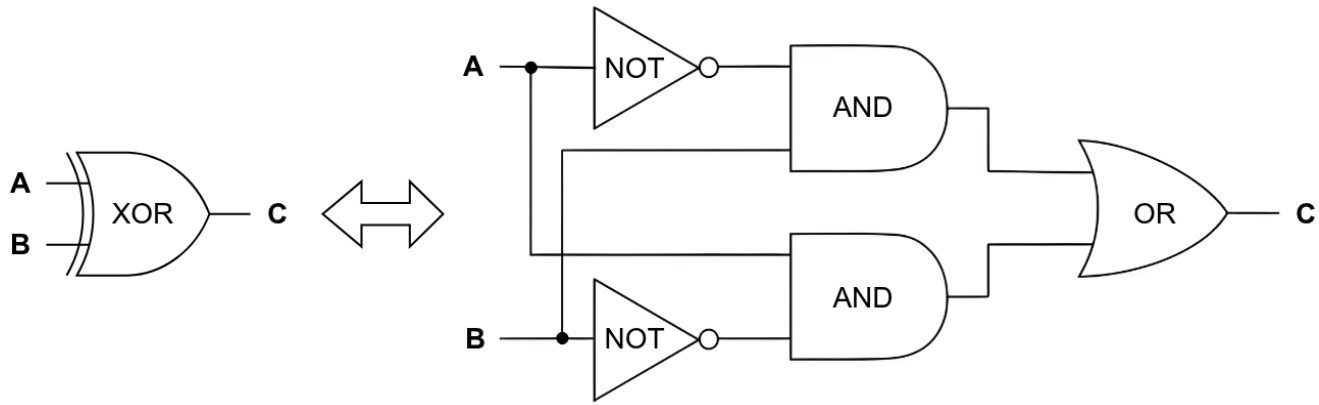
For instance, we see how to implement two common components:

1. **Half Adder**: a circuit that implements the addition of two binary numbers.
2. **Full Adder**: a circuit that implements the addition of two binary numbers with a carry input.

These components can be find on any computer, and are used to perform the addition of two binary numbers.

XOR gate

To easily implement an adder, we need to implement an XOR gate:



As seen in the previous lecture, a XOR can be implemented by combining AND, OR, and NOT gates:

XOR(A, B) = OR(AND(A, NOT(B)), AND(NOT(A), B))

Half Adder: boolean function

We want to design a circuit that:

- camputes the sum of two binary numbers A and B
- the output is:
    - SUM(A, B): the sum of A and B
    - CARRY(A, B): the carry of SUM(A, B)

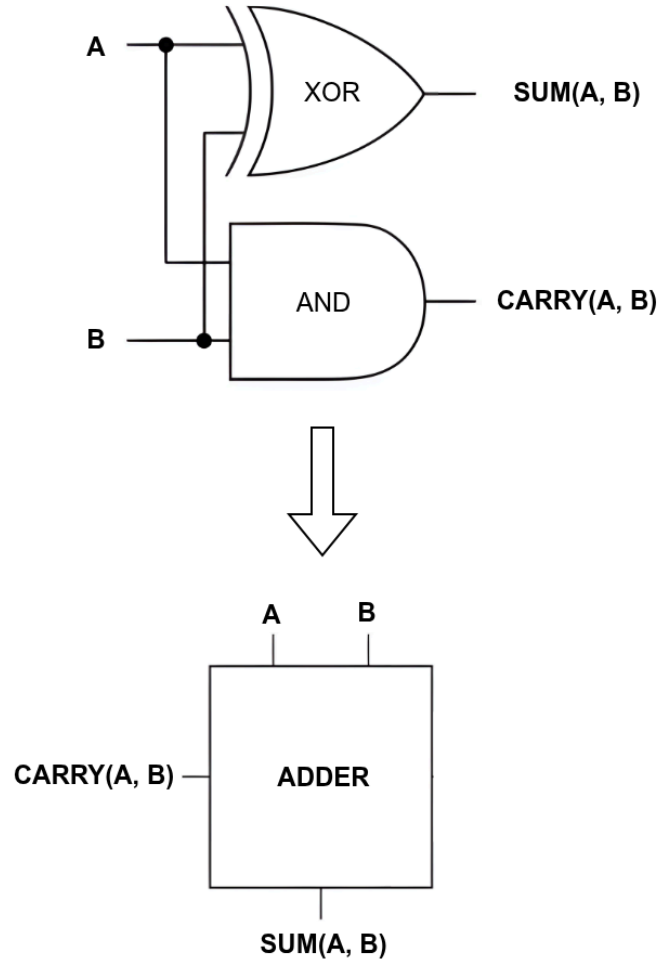
A	B	SUM(A, B)	CARRY(A, B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

We can easily observe by looking at the table above that:

- SUM(A, B) = A XOR B
- CARRY(A, B) = A AND B

NOTE: The formula has two outputs because when both A and B are 1, a single output bit would not enough to represent the sum which would be $1_2 + 1_2 = 10_2$, hence, the output is split into two bits: the sum and the carry.

Half Adder: circuit implementation

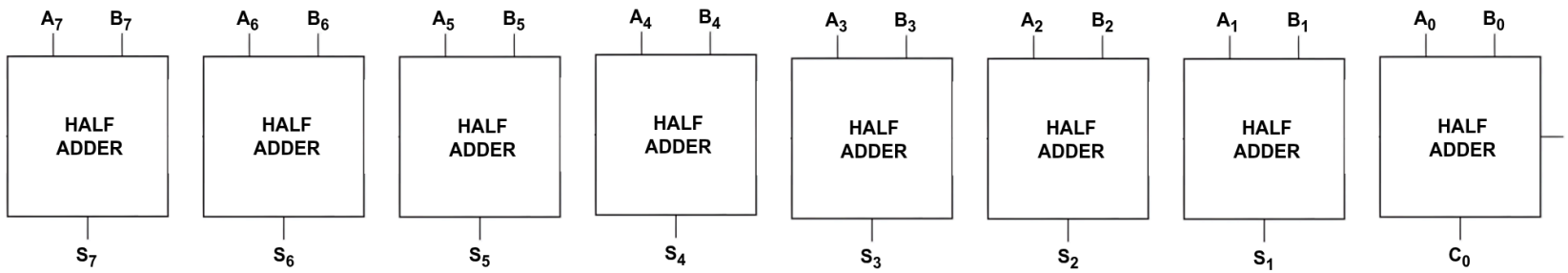


Half Adder: cannot be chained to sum bytes

In the real world, we would to sum two binary numbers A and B that are more than one bit long. For instance:

$$A = 100011100_2 \qquad B = 101010111_2 A + B = ?$$

One strategy could be to split the numbers into bits and sum them pair by pair using half adders:



where A_i, B_i, and S_i are the i-th bit of A, B, and S (the sum) respectively.

PROBLEM: We are ignoring the carry bit of each operation! The result is wrong!

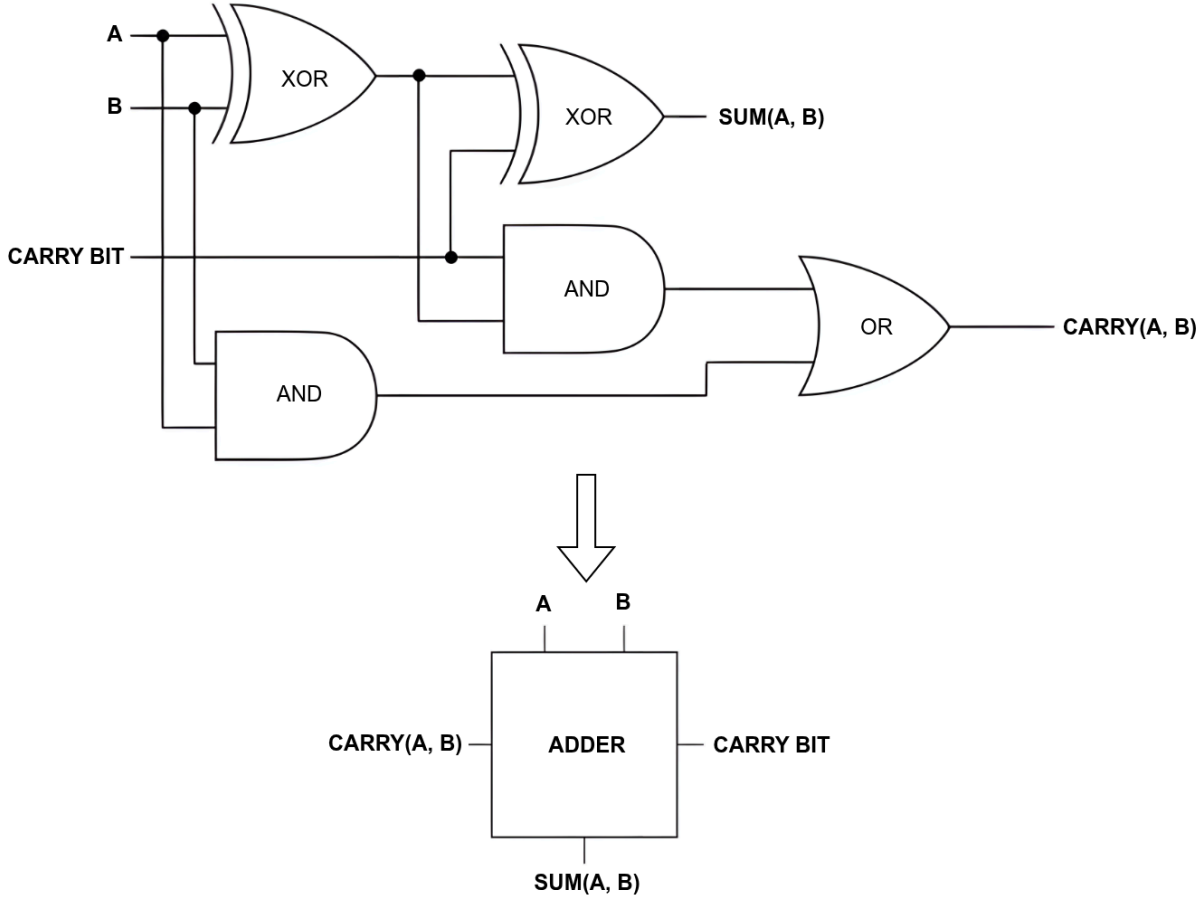
Full Adder: boolean function

We want to design a circut that:

- camputes the sum of:
    - two bit A and B
    - a carry bit (from the previous addition)
- the output is:
    - SUM(A, B, CARRY): the sum of A, B, and CARRY
    - CARRY(A, B, CARRY): the carry of SUM(A, B, CARRY)

A	B	CARRY	SUM(A, B, CARRY)	CARRY(A, B, CARRY)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Adder: circuit implementation

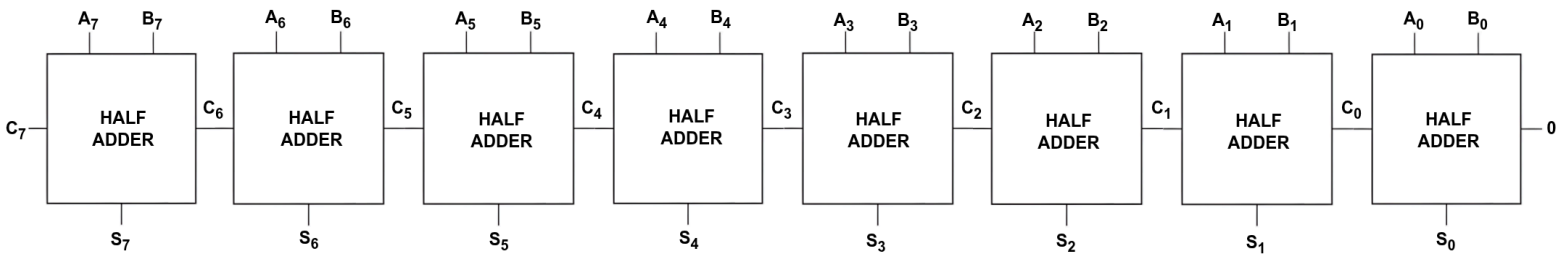


Full Adder: summing two bytes

We now sum two bytes A and B, such as:

$$A = 100011100_2 B = 101010111_2 A + B = ?$$

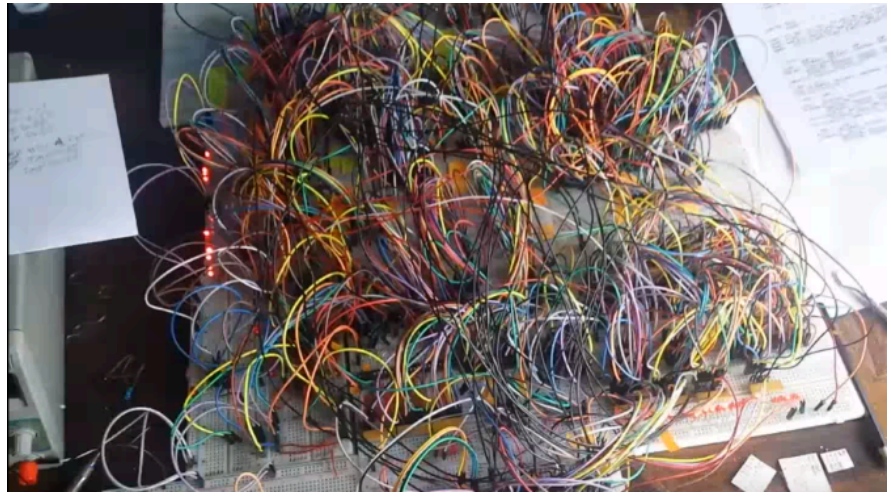
We split the numbers into bits and sum them pair by pair using different full adders:



where A_i, B_i, and S_i are the i-th bit of A, B, and S (the sum) respectively. The first input carry is set to 0, and the last output carry is ignored.

Next: we will implement a full 8-bit computer!

We will use starter kit electronics. Spoiler of the result:



What you need: 7899 transistors, 12786 jump wires, **...just kidding :)**

NOTE: someone has actually done it: see here and here.

How today's computers are built?

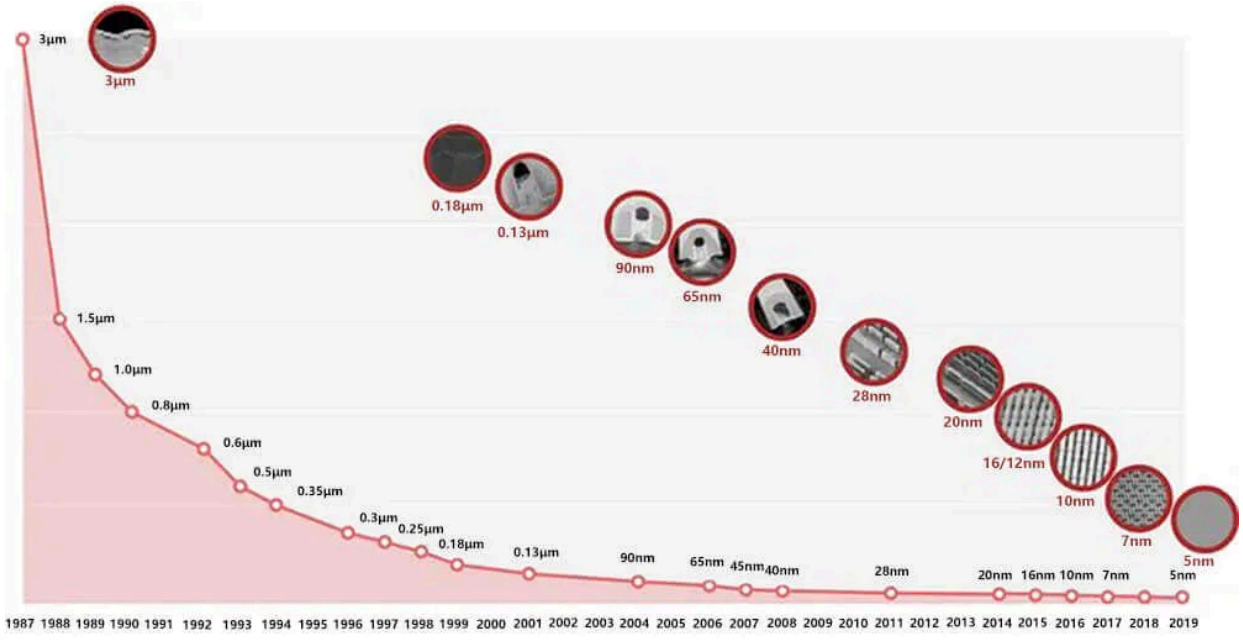
- They are based on the same ideas presented in our lectures...
- ...manufacturers are implementing millions of boolean operations
- ...to do that they need billions of gates
- ...since gates are mostly implemented using transistors
- ...they need **billions of transistors**
- ...which has to be packed in a limited surface to minimize signal propagation

They cannot use standard electronic components. They must reduce their size:

Acronym	Name	Year	Transistor count[91]	Logic gates number[92]
SSI	*small-scale integration*	1964	1 to 10	1 to 12
MSI	*medium-scale integration*	1968	10 to 500	13 to 99
LSI	*large-scale integration*	1971	500 to 20 000	100 to 9999
VLSI	*very large-scale integration*	1980	20 000 to 1 000 000	10 000 to 99 999
ULSI	*ultra-large-scale integration*	1984	1 000 000 and more	100 000 and more

...in the last 20 years they have been able to pack billions of transistors in a chip.

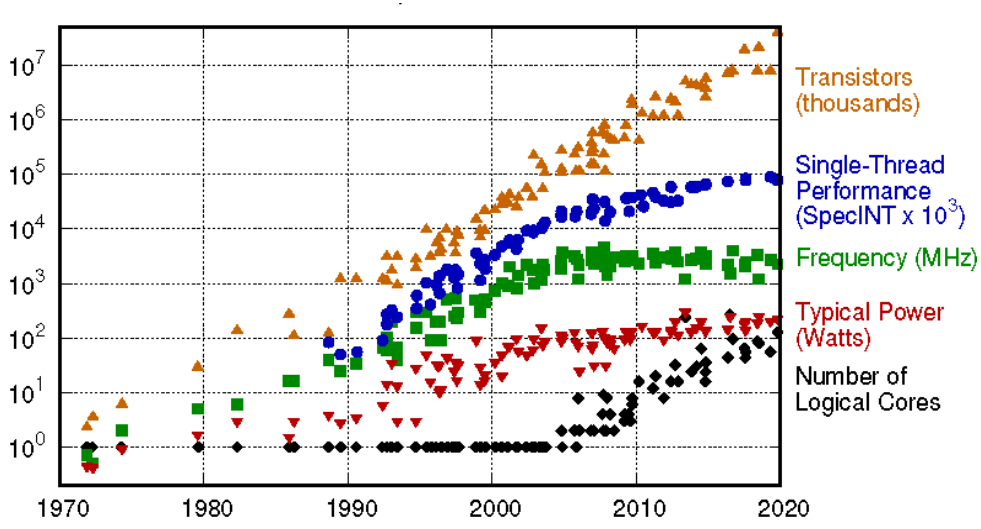
Transistor size evolution



Only a few companies can produce transistors smaller than 10nm:

- TSMC is leading the market: used by AMD, Apple, Samsung, Huawei, etc.
- Intel is struggling to scale under 10nm transistors
- Samsung is also struggling to scale under 10nm transistors

Moore's Law



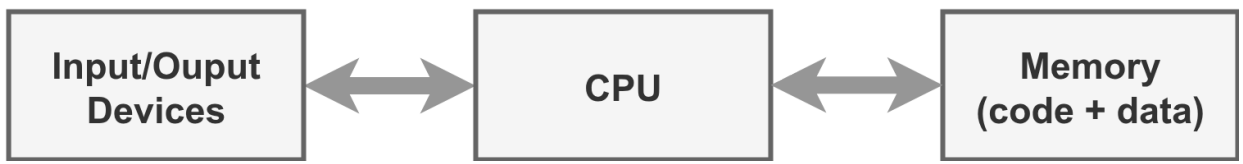
Gordon Moore (Intel): *the number of transistors in an integrated circuit doubles about every two years.*

Before 2010, more transistors translated into faster CPU. Software was getting faster without doing nothing... it was enough to buy a new CPU every year!

After 2010, more transistors translated into more cores or more cache. Unfortunately, the ***free lunch is over***: software is not getting faster without doing nothing... software must be re-engineered to take advantage of more cores or more cache.

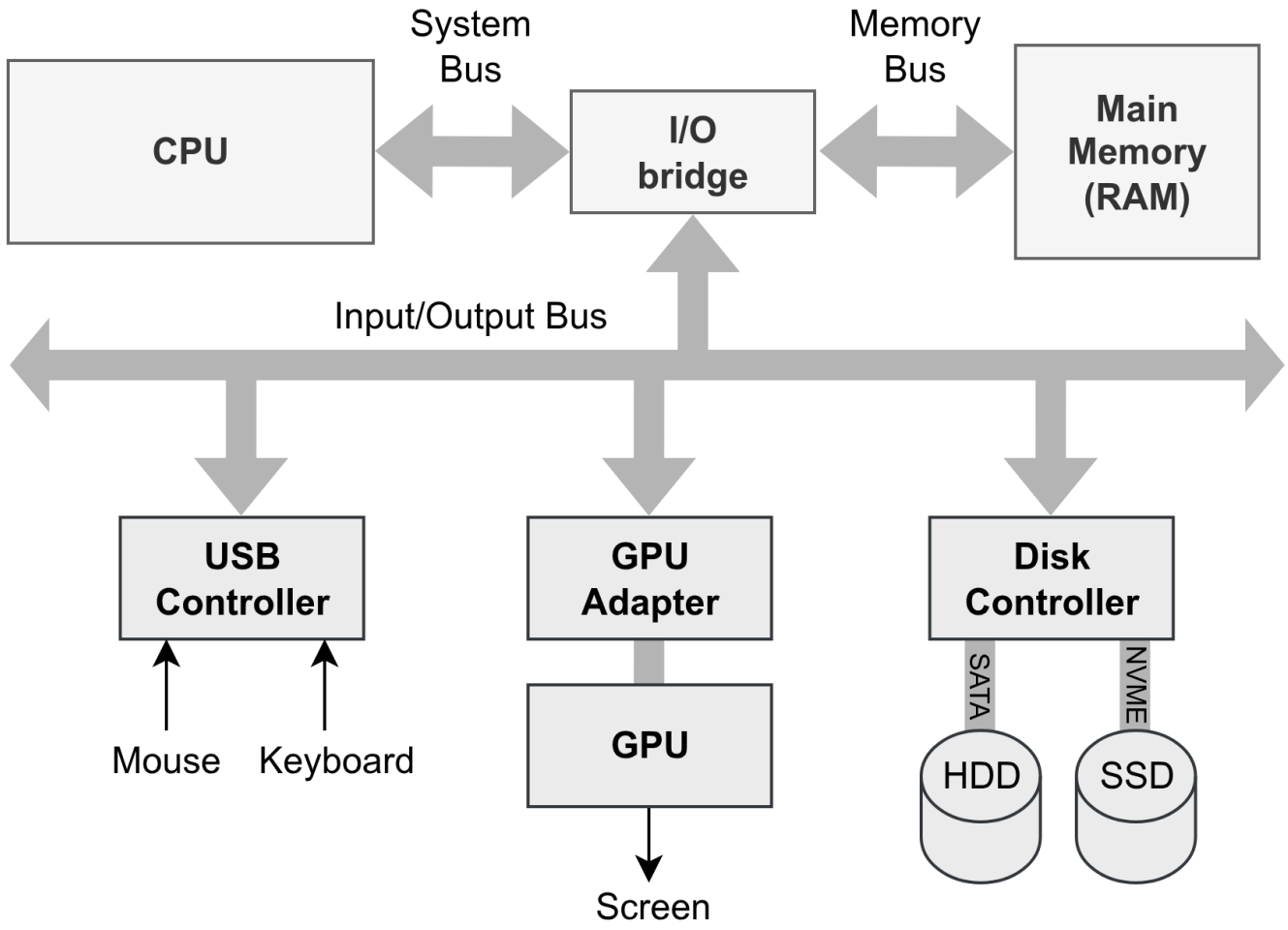
What is inside a computer?

An (still accurate) high level overview of a computer:

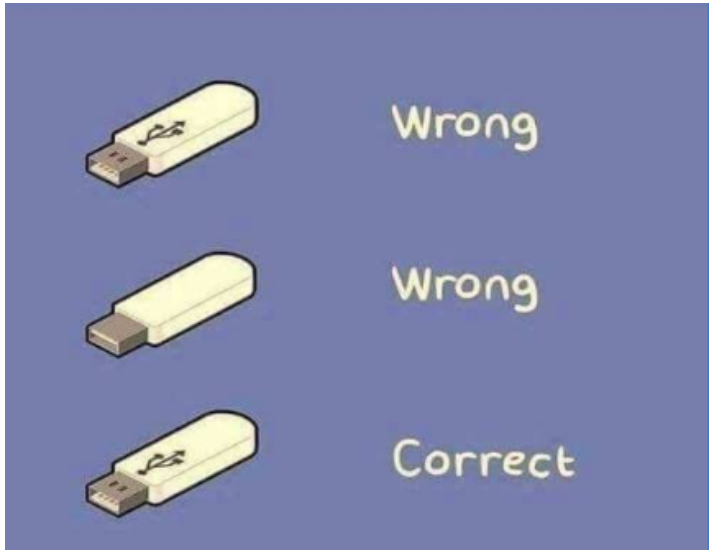


von Neumann architecture (1945)

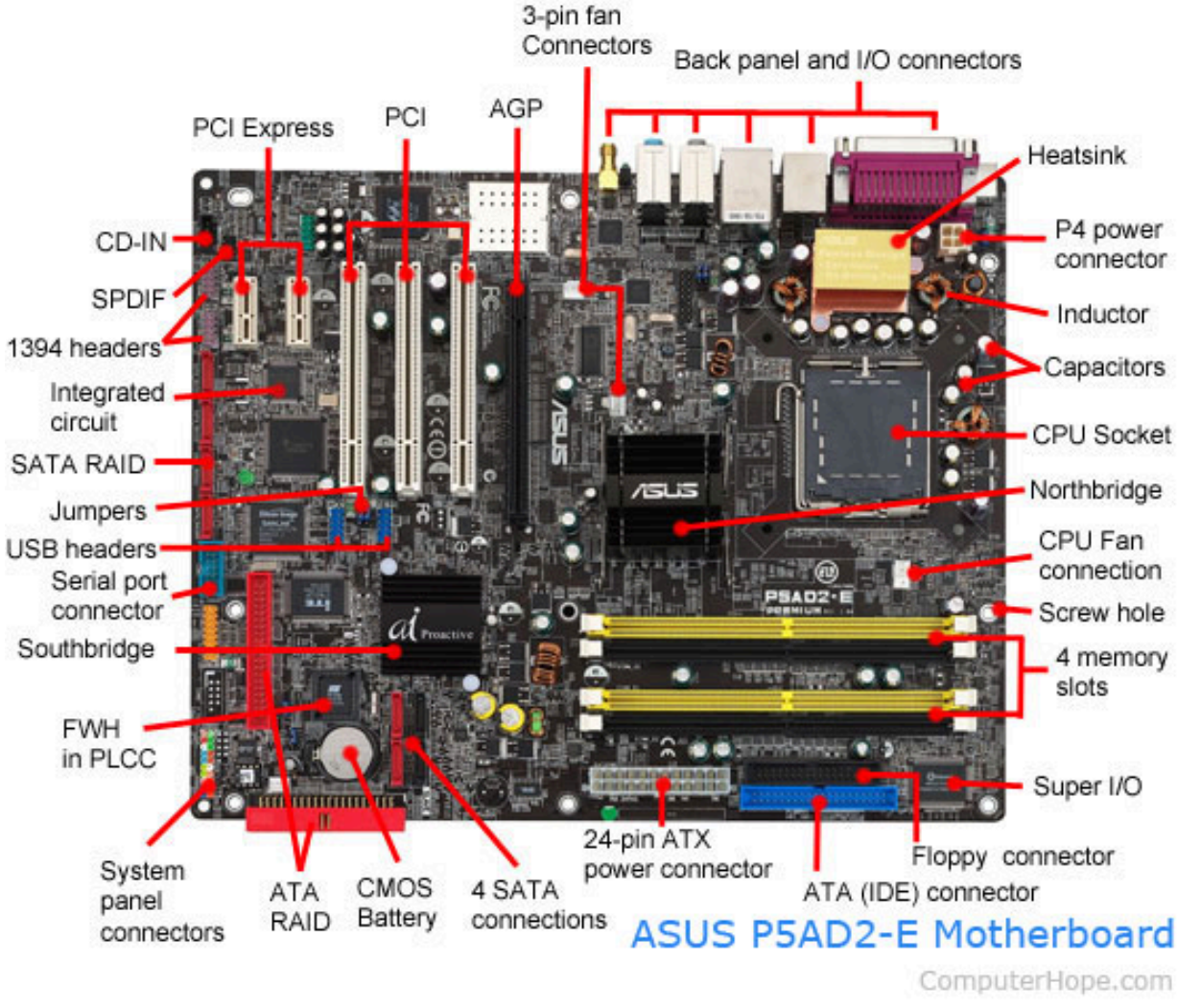
A more detailed overview of a computer



We all know the hardest part...

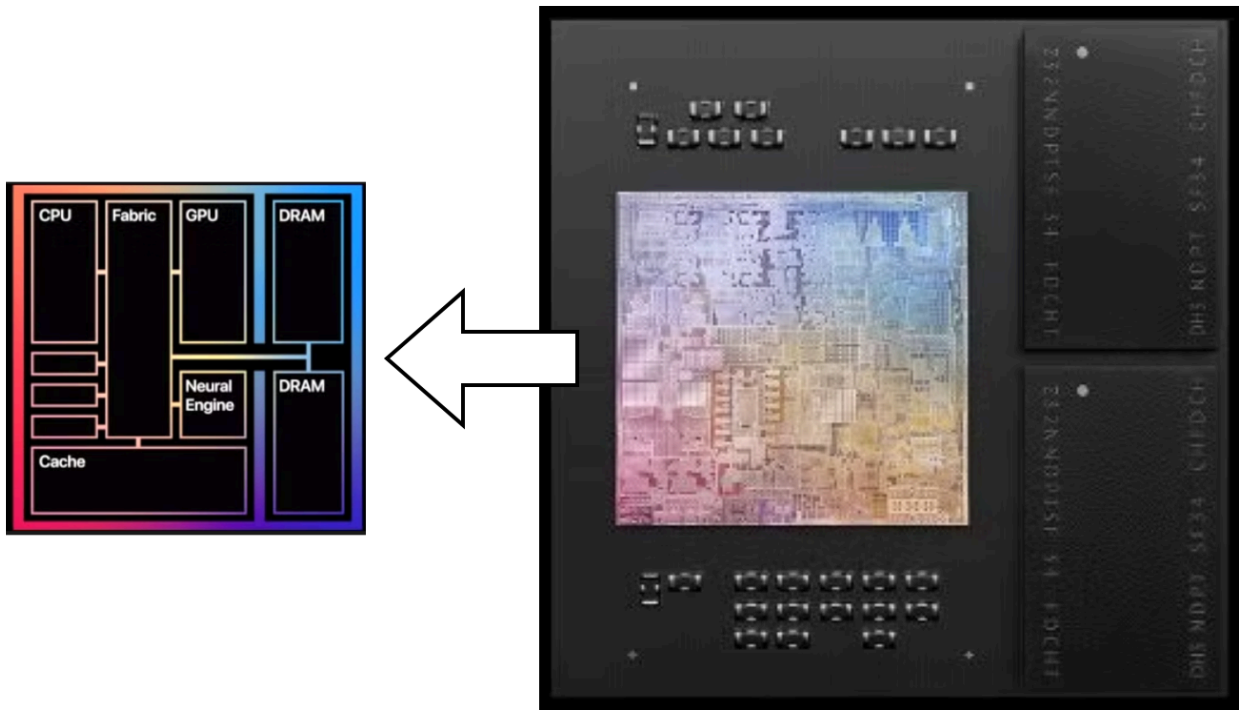


Motherboard of a computer

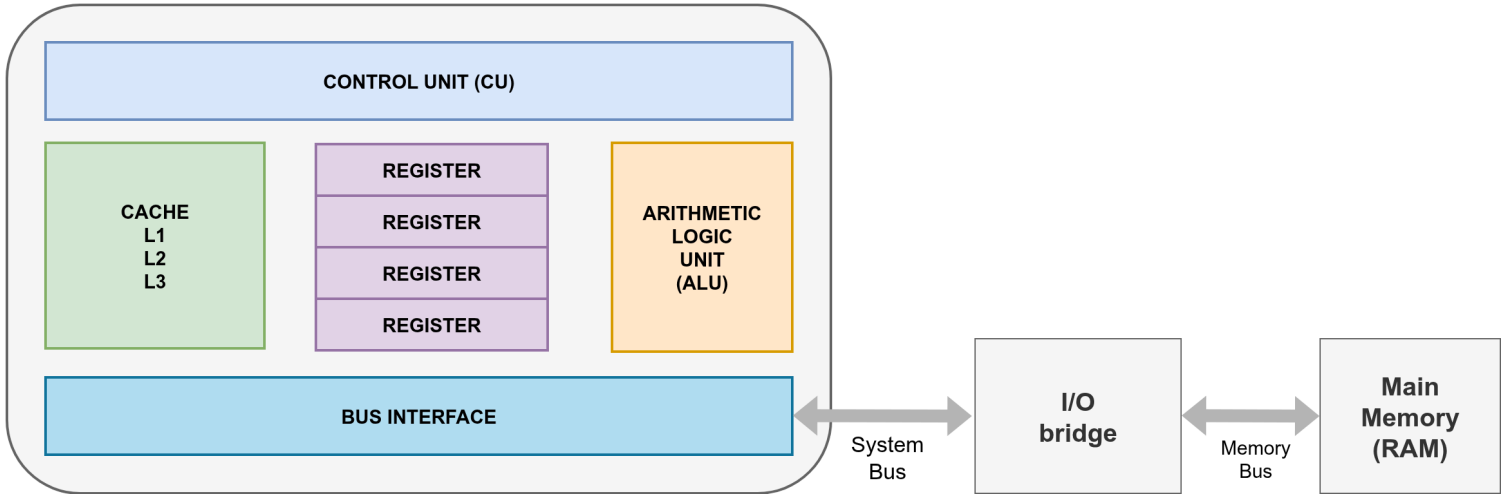

3-pin fan Connectors
Back panel and I/O connectors
PCI Express
PCI
AGP
Heatsink
CD-IN
P4 power connector
SPDIF
Inductor
1394 headers
Capacitors
Integrated circuit
CPU Socket
SATA RAID
Northbridge
Jumpers
USB headers
CPU Fan connection
Serial port connector
Screw hole
Southbridge
4 memory slots
FWH in PLCC
Super I/O
System panel connectors
ATA RAID
CMOS Battery
4 SATA connections
24-pin ATX power connector
Floppy connector
ATA (IDE) connector
ASUS P5AD2-E Motherboard
ComputerHope.com

Modern System On Chip (SoC)

Modern SoCs integrate the CPU, GPU, disk, and other components on a single chip:



CPU (**C**entral **P**rocessing **U**nit)



- The **Arithmetic Logic Unit (ALU)** performs arithmetic operations (such as addition and subtraction) and logical operations (like AND, OR, NOT) on binary data.

- The **Control Unit** coordinates the operations of the CPU, managing the flow of data and instructions (e.g., determining the next instruction to execute, performing the fetch-decode-execute cycle).

- The **Registers** are a few (less than 10-20) small (up to 64 bits wide), fast (<0.5 nanosecond) volatile memory locations that store data and instructions for processing. Most CPUs have to first load data from RAM (~100 nanoseconds) into registers before processing it with the ALU.

- The **Cache** is a high-speed (1-16 nanoseconds) but still limited (e.g., 1-16 MB) volatile memory that stores frequently used data to speed up access.

RAM (**R**andom **A**ccess **M**emory)

- The main (volatile) memory of the computer, where data and instructions are stored for processing before being loaded into the CPU.
- Implemented with Dynamic Random Access Memory (DRAM) chips, based on capacitors (require regular refreshes to maintain data).
- The capacity of RAM is nowadays in order of gigabytes (GB) or terabytes (TB).
- Extremely faster ($1000 - 100000\times$) than secondary storage (e.g., hard drives, SSDs) but significantly slower ($10 - 100\times$) than CPU registers.
- Like a huge array of consecutive bytes, each identified by its address (e.g., $0, 1, 2, \ldots, 2^{64} - 1$).

Disks

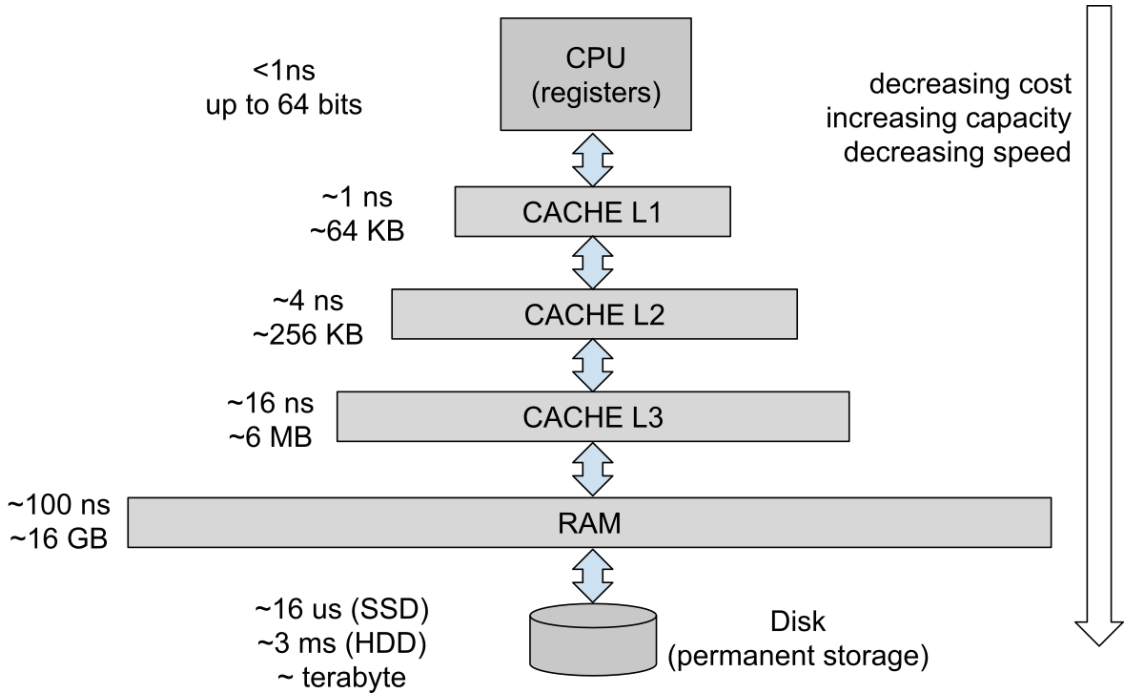
Permanent storage devices used to store data and programs for long-term use:

- **Hard Drives (HDDs)**: use spinning disks to store data, quite slow speed (~200 MB/s) but large capacity (up to 20 TB).
- **Solid-state disks (SSDs)**: use flash memory to store data, much faster speed (~1-4 GB/s) but smaller capacity (up to 2 TB).

Disks can be connected to the computer in different ways:

- USB (Universal Serial Bus): used for external disks.
- IDE (Integrated Drive Electronics): used for mechanical disks.
- SATA (Serial Advanced Technology Attachment): used for mechanical disks and older SSDs.
- NVMe (Non-Volatile Memory Express): used for modern SSDs.

Memory hierarchy



Fast memory is expensive, hence a complex hierarchy is used to compromise between cost and performance: larger data is stored in slower, cheaper memory, that, when needed, is loaded into faster, more expensive memory into smaller amounts.

Cloud...?

How can program a computer?

In the next lecture, we will cover the software side of the story.